

ELT-Bench-Verified: Benchmark Quality Issues Underestimate AI Agent Capabilities

Christopher Zanolì
IBM Research
ETH Zurich
Zurich, Switzerland
Christopher.Zanolì@ibm.com

Andrea Giovannini
IBM Research
Zurich, Switzerland
AGV@zurich.ibm.com

Tengjun Jin
University of Illinois (UIUC)
Urbana, USA
tengjun2@illinois.edu

Ana Klimovic
ETH Zurich
Zurich, Switzerland
aklimovic@ethz.ch

Yotam Perlitz*
IBM Research
Zurich, Switzerland
Y.Perlitz@ibm.com

ABSTRACT

Constructing Extract-Load-Transform (ELT) pipelines—the backbone of modern data integration—remains one of the most labor-intensive tasks in data engineering, making it a high-impact target for AI-driven automation. On ELT-Bench, the first and only benchmark for evaluating AI agents on end-to-end ELT pipeline construction, agents achieved low success rates that suggested they are far from practical utility. We revisit these results and identify two factors that led to a substantial underestimation of agent capabilities. First, re-evaluating ELT-Bench with the same agent framework but upgrading only the underlying large language model reveals that the extraction and loading stage is largely solved, while transformation performance improves dramatically. Second, we develop an *Auditor-Corrector* methodology that combines scalable LLM-driven root-cause analysis with rigorous human validation (inter-annotator agreement Fleiss’ $\kappa = 0.85$) to systematically audit benchmark quality. Applying this methodology to ELT-Bench uncovers that the majority of failed transformation tasks contain benchmark-attributable errors—including overly rigid evaluation scripts, ambiguous specifications, and incorrect ground truth—that penalize correct agent outputs. Based on these findings, we construct ELT-Bench-Verified, a revised benchmark with refined evaluation logic and substantial ground-truth revisioning. Re-evaluating on ELT-Bench-Verified yields a significant further improvement attributable entirely to benchmark correction. Our results demonstrate that both rapid model improvement and benchmark quality issues contributed to a substantial underestimation of agent capabilities in the original evaluation. More broadly, our findings echo recent observations of pervasive annotation errors in text-to-SQL benchmarks, suggesting that benchmark quality issues are a systemic problem across data engineering evaluation. Systematic quality auditing should become standard practice, especially for benchmarks that evaluate complex, multi-step agentic tasks. We release ELT-Bench-Verified as a community resource to provide a more reliable foundation for measuring and driving further progress in AI-driven data engineering automation.

PVLDB Reference Format:

Christopher Zanolì, Andrea Giovannini, Tengjun Jin, Ana Klimovic, and Yotam Perlitz. ELT-Bench-Verified:

*Corresponding author.

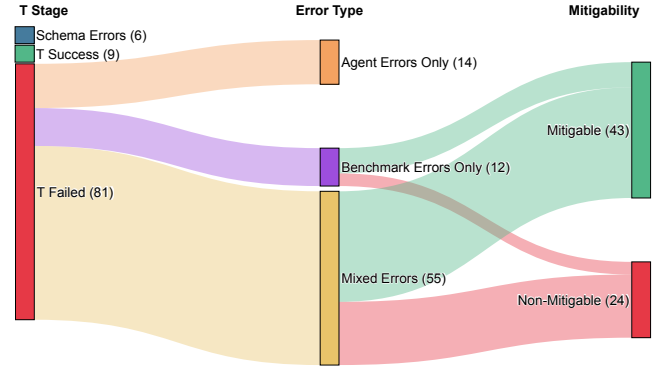


Figure 1: Distribution of error attribution across 81 failed transformation tasks in ELT-Bench. Tasks are classified by error source—agent-attributable, benchmark-attributable, or mixed—and further stratified by mitigability, distinguishing errors addressable through evaluation refinements from those requiring ground truth column removal.

Benchmark Quality Issues Underestimate AI Agent Capabilities. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/uiuc-kang-lab/ELT-Bench/pull/18>.

1 INTRODUCTION

Modern organizations rely heavily on Extract-Load-Transform (ELT) pipelines—workflows that extract data from heterogeneous sources,

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

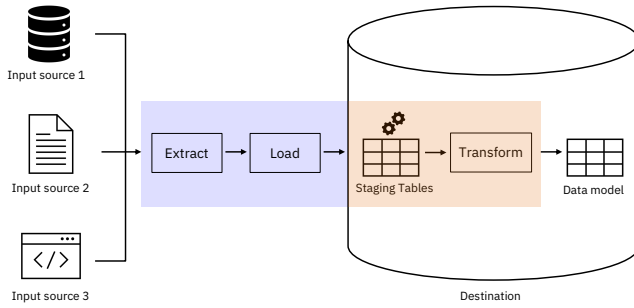


Figure 2: Overview of a typical ELT pipeline. Data is first extracted from heterogeneous sources—such as databases, APIs, flat files—using connectors like Airbyte [1]. It is then loaded in raw form into a cloud data warehouse such as Snowflake [7]. Finally, transformation logic, typically written as SQL models in dbt [8], reshapes the raw data into analytics-ready tables within the warehouse.

load it into a cloud warehouse, and transform it in place (Figure 2)—to integrate and analyze data at scale [11].

However, constructing these pipelines remains a highly manual, labor-intensive process requiring expertise across diverse data sources [2, 17], cloud warehouses like Snowflake [7], and transformation frameworks like dbt [8]. The prospect of automating this workflow with AI agents is a major goal for data engineering. To measure progress toward this goal, ELT-Bench [10] was recently introduced as the first benchmark for end-to-end ELT pipeline construction. Yet, the initial baseline results were stark: SWE-Agent with Claude Sonnet 3.5 [4] achieved only a 37% success rate on data extraction and loading and a mere 1% on data transformation. These figures suggested that current AI agents are far from practical utility in real-world data engineering tasks.

We revisit these findings and show that this assessment was premature, driven by two compounding factors. First, the original evaluation captured a snapshot of older capabilities. By upgrading only the underlying model to Claude Sonnet 4.5 [6] while keeping the same agent framework (SWE-Agent [22]), we observe a substantial leap in performance: extraction and loading rises from 37% to 96%—indicating that this phase is largely solved for the source types and tool configurations covered by ELT-Bench—while the transformation success rate jumps from 1% to 22.66%.

Despite this substantial improvement, a 22.66% transformation success rate appears surprisingly low given that modern approaches to related tasks such as text-to-SQL generation now exceed 81% execution accuracy [18, 20]. This gap led us to question whether we had reached the ceiling of the agent’s capabilities, or the ceiling of the benchmark’s validity. Motivated by recent discoveries of pervasive annotation errors in text-to-SQL benchmarks [9], we conduct a systematic data quality audit of ELT-Bench. Of the 100 benchmark tasks, 96 pass extraction and loading with the upgraded model; of those, 87 fail the transformation stage, and 81 produce transformation outputs yet fail column-level evaluation—these 81 tasks form the scope of our audit (Figure 4). Through a rigorous process

validated by three independent annotators (Fleiss’ $\kappa = 0.85$), our investigation uncovers pervasive evaluation issues: 82.7% of these 81 tasks contain at least one benchmark-attributable error. At a finer granularity, each task comprises multiple columns that are evaluated independently; at this column level, 33.0% of all mismatches are benchmark-attributable rather than genuine agent failures. These errors range from ambiguous task specifications and overly rigid evaluation scripts to mathematically incorrect ground-truth values (Table 3, Figure 1).

To establish a reliable measure of AI capabilities in data engineering, we introduce a two-stage **Auditor-Corrector** framework (Figure 5)—a general, human-validated methodology for auditing complex data engineering benchmarks at scale. Our *Auditor* pipeline combines LLM-driven root-cause analysis with comprehensive manual verification: it systematically reverse-engineers the correct SQL for each unmatched column and classifies every mismatch by attribution, with all findings reviewed by domain experts and validated through an inter-annotator agreement study (Fleiss’ $\kappa = 0.85$). Our *Corrector* pipeline then translates these findings into targeted benchmark refinements to construct ELT-Bench-Verified. By re-evaluating the exact same agent and model on this verified benchmark, the transformation success rate rises from 22.66% to 32.51%—a 43.5% relative improvement attributable entirely to benchmark correction. In summary, our core contributions are:

- (1) **Updated EL Stage Baseline:** We show that extraction and loading is largely solved for ELT-Bench’s covered configurations, with SRDEL rising from 37% to 96% by upgrading only the underlying model.
- (2) **An Auditor-Corrector Methodology:** We develop a scalable methodology for auditing execution-based benchmarks, combining LLM-driven root-cause analysis with structured human verification. Applied to ELT-Bench, it classifies 660 column-level mismatches into a novel error taxonomy, revealing that 82.7% of failed tasks contain benchmark-attributable errors.
- (3) **ELT-Bench-Verified:** We construct a corrected benchmark by refining evaluation scripts and removing unreliable ground-truth columns, raising SRDT from 22.66% to 32.51% with the same agent and model, and release it as a community resource for reliable evaluation of AI agents on ELT tasks.

2 REVISITING ELT-BENCH

ELT-Bench [10] is the first benchmark for evaluating AI agents on end-to-end ELT pipeline construction. It comprises 100 tasks spanning diverse databases, each requiring the agent to build a complete pipeline from a provided starter codebase containing connection configurations, target data model specifications, source table schemas, and data tool documentation. Figure 3 illustrates the two-stage agent workflow. Performance is measured by two metrics: SRDEL (Success Rate for Data Extraction & Loading), the fraction of tasks where all source data is correctly loaded, and SRDT (Success Rate for Data Transformation), the fraction of total target data models whose columns match the ground truth.

Before auditing benchmark quality, we first establish an updated performance baseline by re-evaluating ELT-Bench with a more recent model than those used in the original study.

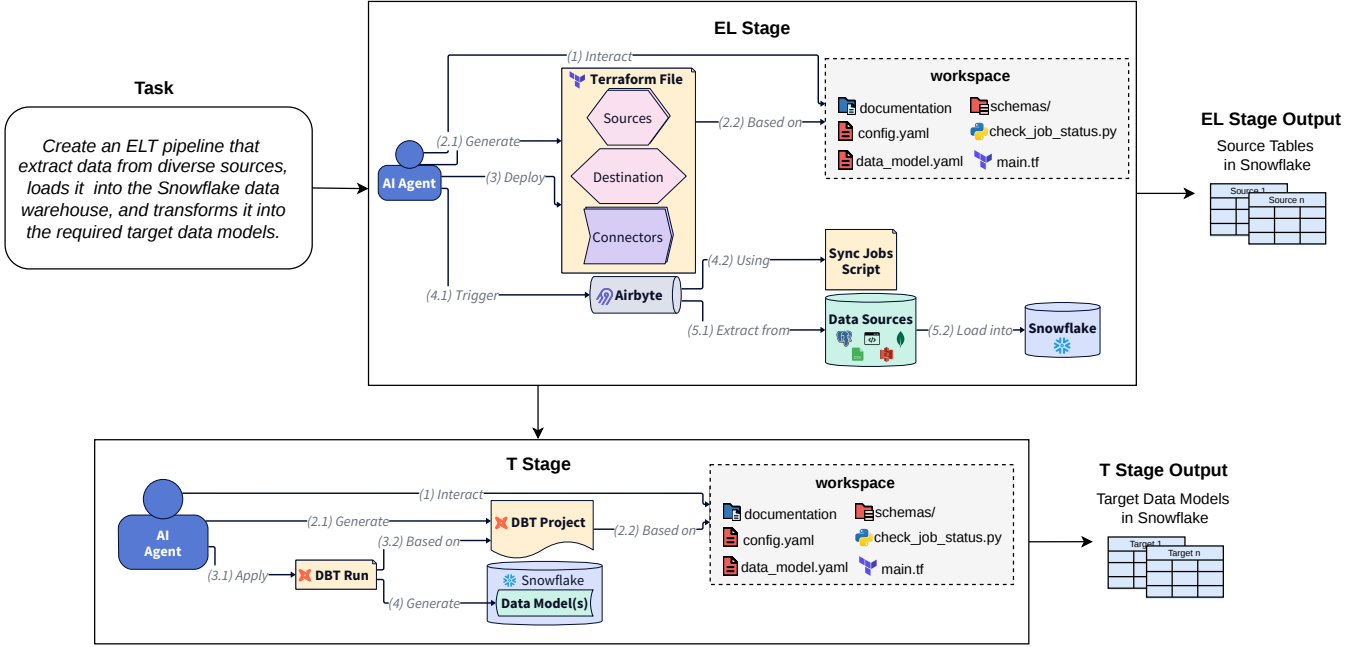


Figure 3: Agent workflow in ELT-Bench, illustrating the two-stage pipeline the agent must construct. **EL Stage:** The agent reads connection details from `config.yaml`, writes Terraform configurations to define Airbyte [1] sources, the Snowflake [7] destination, and their connectors, deploys the configuration, and triggers synchronization jobs that extract data from heterogeneous sources (databases, APIs, flat files) and load it into the warehouse as staging tables. The agent then uses a provided Python script in the workspace to poll job status and proceeds to the T stage once all jobs succeed (i.e., all source tables for the task have been loaded into Snowflake). **T Stage:** The agent initializes a dbt [8] project, authors SQL transformation models that reshape the staged data into the target data models defined in `data_model_schema.yaml`, and executes `dbt run`, which generates the target data model(s) in the Snowflake data warehouse.

2.1 Experimental Setup

We evaluate SWE-Agent [22] with Claude Sonnet 4.5 across all 100 ELT-Bench tasks, using the same evaluation protocol and metrics (SRDEL and SRDT) defined by the original authors. We select SWE-Agent because it was one of two agent frameworks evaluated in the original ELT-Bench study, enabling direct comparison. We focus on a single model upgrade rather than evaluating multiple model-agent combinations due to the substantial computational cost of each full evaluation run: a single SWE-Agent run across all 100 tasks requires 2 days, 7 hours and 16 minutes of wall-clock time and costs \$343 in API fees, while a Baseline (ReAct) run requires 18 hours and 17 minutes at \$293. The cumulative cost of the two runs reported in this paper alone exceeds 3 days and \$635.

2.2 Results

Figure 6 presents our results alongside the original ELT-Bench findings, including the effect of benchmark correction (ELT-Bench-Verified, detailed in Section 5). In terms of data models passed, the original model achieves 2/203, the upgraded model 46/203, and the upgraded model on ELT-Bench-Verified 66/203.

The data extraction and loading stage shows a 59 percentage point improvement (37% to 96%), with 96 of 100 tasks successfully loading data into Snowflake. Since the agent framework is held constant, this improvement is attributable to the model upgrade. This

result suggests that, for the source types and tool configurations covered by ELT-Bench, the extraction and loading stage is largely solved.

The transformation stage improves from 1% to 22.66% SRDT. Of the 96 successfully loaded tasks, 9 pass the transformation stage (accounting for 46 of 203 data models), 87 fail, and 6 of those 87 fail due to schema errors—non-existent tables caused by premature agent termination—leaving 81 tasks that produce transformation outputs but fail column-level evaluation. Including the 4 tasks that failed extraction entirely, 77.3% of all data models (157/203) do not pass transformation evaluation.

This raises a natural question: to what extent do these remaining failures reflect genuine agent limitations versus benchmark quality issues? We investigate this in the following sections.

3 METHODOLOGY

To audit and refine ELT-Bench, we introduce a two-stage *Auditor-Corrector* framework. The Auditor reverse-engineers the correct SQL for each unmatched column and classifies every mismatch by attribution; the Corrector translates these findings into targeted benchmark refinements. Figure 5 provides an overview.

Formal setup. ELT-Bench comprises N database tasks $d_1, \dots, d_N$. Each task d_i contains one or more target data models, and each data

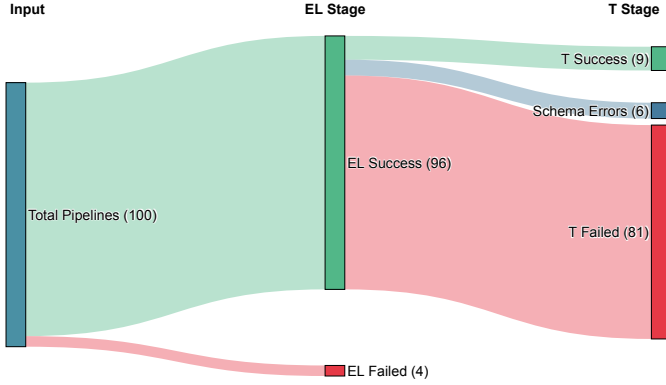


Figure 4: Pipeline outcomes across extraction/loading and transformation stages.

model defines a set of target columns. We write C_i for all columns in task d_i . The evaluation script compares each predicted column against its ground truth counterpart and marks it as *matched* or *unmatched*. We define $\mathcal{U} \subseteq \bigcup_i C_i$ as the set of all unmatched columns across all tasks. Our framework operates exclusively over $\mathcal{U}$. In our instantiation, $N = 81$ tasks (those that produced transformation outputs but failed evaluation), yielding $|\mathcal{U}| = 660$ unmatched columns distributed across 136 data models.

3.1 The Auditor Pipeline

The Auditor combines automated analysis with systematic manual verification to reverse-engineer the correct SQL for each unmatched column and classify every mismatch. It proceeds in three phases.

3.1.1 Phase 1: Analysis Environment Preparation. For each task d_i containing at least one unmatched column, we construct a structured analysis environment populated with the artifacts necessary for root cause analysis: the task specification (`data_model.yaml`), source tables and schemas, ground truth tables (`gt/<data_model>.csv`), and agent-generated SQL and predictions (`predicted/<data_model>.sql`, `predicted/<data_model>.csv`). The environment is populated selectively: tasks where all columns match are excluded entirely, and within each task, only data models containing at least one unmatched column are included. This filtering reduces the analysis scope from the full benchmark ($N = 100$ tasks, 203 data models) to the relevant subset ($N = 81$ tasks, 136 data models, 660 columns).

3.1.2 Phase 2: Scaled Analysis via LLM Agent. We adopt an agentic approach in which an LLM-powered agent (Claude Opus 4.5 [5]) is instantiated within each task’s analysis environment and tasked with diagnosing every column-level mismatch. Analysis is parallelized at the task level: one agent instance is spawned per task d_i , with full read access to all task artifacts. The agent is guided by a two-level prompt architecture: an *orchestration prompt* (`prompt_all.md`) parses the evaluation log, builds a work queue of (database, table, columns) tuples, and dispatches each item to an *analysis prompt* (`prompt.md`) that encodes the per-column analytical protocol. Both prompts are released as supplementary material.

For each unmatched column $c \in \mathcal{U} \cap C_i$, the agent autonomously navigates the task environment—inspecting the column specification, tracing the relevant SQL logic in the agent’s predicted query, comparing predicted and ground truth values, and reasoning over the observed discrepancies. The agent iteratively formulates hypotheses about the mismatch root cause and tests them by deriving alternative SQL interpretations and executing each against the source data to compare with the ground truth. For each column, the agent produces a structured analysis report containing: (i) a root cause diagnosis identifying the specific flaw; (ii) a corrected SQL derivation, *verified* by the agent to achieve a 100% row-level match with the ground truth when executed against the source data; and (iii) supporting evidence including value-level comparisons and match statistics demonstrating the exact match. For the 30 columns where no SQL interpretation achieves a perfect match—indicating that the ground truth values cannot be derived from any reasonable reading of the task specification—the agent explicitly flags the column and reports the closest approximation. In total, this phase produces 660 analysis reports covering all unmatched columns.

3.1.3 Phase 3: Manual Verification and Categorization. A data engineer manually reviews all 660 analysis reports. For each report, the reviewer verifies that (i) the identified root cause is consistent with the presented evidence, (ii) the corrected SQL logic (if proposed) is semantically sound, and (iii) the match statistics are accurate.

Following validation, the reviewer assigns each column to an error category. The resulting categorization assigns each column $c \in \mathcal{U}$ to exactly one error category along two dimensions: *attribution* (agent-attributable vs. benchmark-attributable) and *mitigability* (whether the error can be corrected without modifying ground truth data). The primary categorization is performed by a single data engineer. To validate reproducibility, we conduct an inter-annotator agreement (IAA) study in which three annotators independently classify a uniformly sampled subset of 50 columns using the full 14-category taxonomy, given only the analysis reports and a detailed annotation protocol with category definitions, decision rules and worked examples. At the high level—which directly underpins the headline attribution statistics—Fleiss’ $\kappa = 0.851$ (almost perfect agreement, 94.7%). At exact-category granularity across all 14 taxonomy classes, Fleiss’ $\kappa = 0.755$ (substantial agreement, 79.3%), confirming that the fine-grained categorization is reproducible across annotators.

3.1.4 Validation of the Auditor. The structured prompt instructs the agent to produce, for each unmatched column, a dedicated analysis report and a Python debug script. Because the agent operates autonomously, its intermediate artifacts exhibit variation: debug scripts were sometimes written in SQL rather than Python (9 of 81 tasks), omitted entirely when the root cause was evident from direct inspection (5 tasks), or produced in multiple iterative versions as the agent refined its hypotheses (27 tasks). In 4 tasks where a single systemic root cause affected all columns in a table, the agent produced a combined analysis report rather than separate per-column reports. Despite these deviations, every unmatched column is covered: the 660 columns are addressed across 528 analysis reports, with multi-column reports accounting for the difference. Crucially, for 630 of 660 columns the agent’s corrected SQL is self-validated: the agent programmatically executes the corrected query

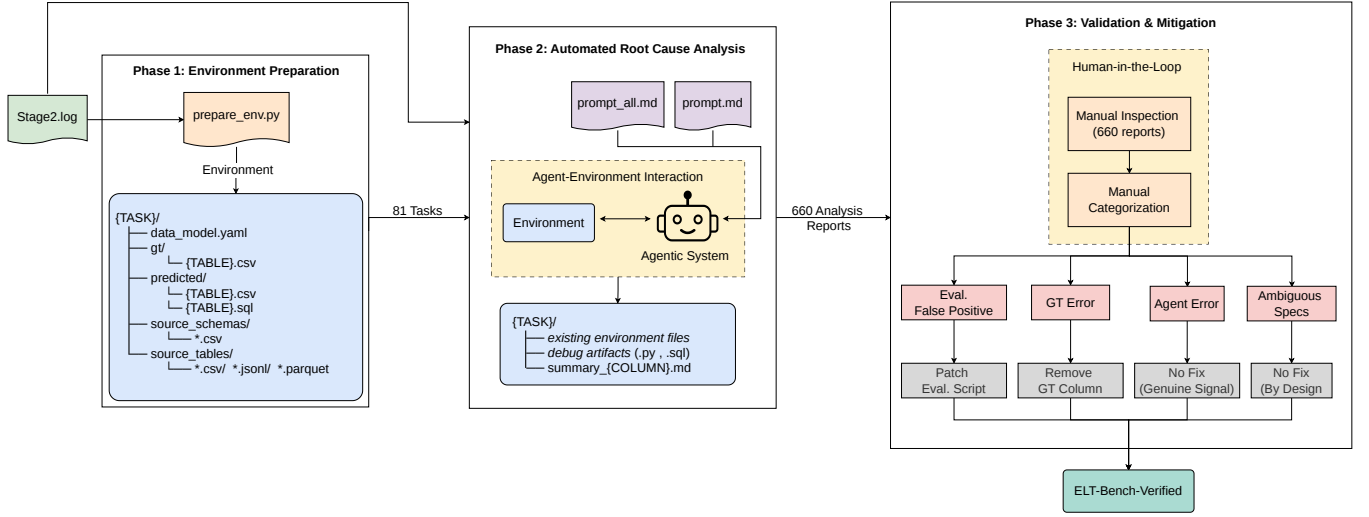


Figure 5: Overview of the Auditor-Corrector framework. The *Auditor* proceeds in three phases: Phase 1 constructs a structured analysis environment for each failed task from the evaluation log; Phase 2 deploys an LLM agent per task that autonomously reverse-engineers the correct SQL for each unmatched column, verifying each derivation against the ground truth; Phase 3 involves manual validation and error categorization of all 660 reports. The *Corrector* applies category-specific corrections—evaluation script refinements and removal of columns with unreliable ground truth—to produce ELT-Bench-Verified.

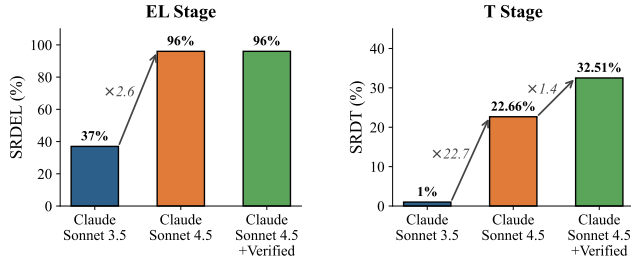


Figure 6: Performance progression across three configurations, all using SWE-Agent: the original model (Claude Sonnet 3.5), the upgraded model (Claude Sonnet 4.5), and the upgraded model evaluated on ELT-Bench-Verified. The EL stage is largely solved after the model upgrade ($\times 2.6$), while the T stage benefits from both model improvement ($\times 22.7$) and benchmark correction ($\times 1.4$).

against the source data and confirms a 100% row-level match with the ground truth before finalizing the report. For the remaining 30 columns, the agent explicitly reports that no SQL interpretation achieves a perfect match, flagging them as potential ground truth errors. A data engineer subsequently reviewed all 660 reports; no report required correction of its core diagnostic content.

3.2 The Corrector Pipeline

The Corrector translates the Auditor’s findings into benchmark refinements without introducing new biases.

3.2.1 Correction Logic. The categorization from Phase 3 classifies each column along two dimensions: *attribution* (benchmark-attributable vs. agent-attributable) and *mitigability*. Benchmark-attributable errors fall into three categories: *Evaluation False Positives* (correct agent outputs penalized by rigid evaluation scripts), *Ground Truth Calculation Errors* (ground truth values that cannot be derived from any reasonable interpretation), and *Ambiguous Data Model Descriptions* (underspecified column definitions). The full taxonomy, including agent-attributable error categories, is presented in Section 4. Based on this categorization, we apply three types of corrections:

- **Evaluation Script Refinements:** For columns classified as Evaluation False Positives, we patch the evaluation script to handle representational equivalences (e.g., boolean normalization, floating-point tolerance, percentage format normalization, NULL equivalence, order-insensitive comparison). Each patch is derived directly from the mismatch pattern documented in the analysis report.
- **Ground Truth Column Removal:** For columns classified as Ground Truth Calculation Errors—where no SQL interpretation achieved a 100% match during Phase 2—we remove the affected columns from the benchmark. An independent validation study (Section 3.2.2) confirms that no reliable correction can be established for these columns, as human experts themselves disagree on the correct interpretation. Only the 30 affected columns are removed; all remaining columns in each data model are preserved.
- **Preservation of Ambiguity:** We deliberately preserve specification ambiguity for columns classified as Ambiguous Data Model Descriptions. Handling real-world uncertainty is a capability that agents should possess, and this

aspect of the benchmark should be evaluated rather than simplified.

Agent-attributable errors receive no correction, as these represent genuine evaluation signal.

3.2.2 Validation of the Corrector. We validate the Corrector along two axes, corresponding to the two types of corrections applied.

Evaluation Script Refinements. To verify that the patched evaluation script aligns with human judgment better than the original, we conduct a stratified annotation study. We run both the original and patched evaluation scripts on the same agent outputs (SWE-Agent with Claude Sonnet 4.5) using the original ground truth, then classify each column into one of three strata based on the two scripts’ labels: (A) both scripts report a match, (B) the original script reports a mismatch but the patched script reports a match (i.e., corrected false positives), and (C) both scripts report a mismatch. We exclude columns associated with tasks that failed during the extraction stage, as well as the 30 columns identified with ground truth errors (i.e., those classified under the *ground truth error* category in Phase 3 of our methodology). This filtering isolates the effect of the script refinements.

After excluding columns from tasks that failed extraction (4 tasks) or encountered schema errors (6 tasks), as well as the 30 ground truth error columns, the eligible population comprises 2,202 columns: 1,572 in Stratum A, 156 in Stratum B, and 474 in Stratum C. We sample 50 columns using stratified sampling: 15 from Stratum A, 20 from Stratum B, and 15 from Stratum C, deliberately oversampling Stratum B (the corrected false positives) to ensure sufficient statistical power on the decisive stratum where the two scripts differ. For each sampled column, three engineers independently label each column pair as *semantically equivalent* or *not equivalent*, given the ground truth values, the agent’s predicted values, and the column’s natural language description, without knowledge of which stratum the column belongs to or what either script reported. We measure inter-annotator agreement using Fleiss’ κ , obtaining $\kappa = 0.664$ (substantial agreement), with 86% of columns receiving a unanimous label. We resolve disagreements by majority vote and use the resulting consensus labels as ground truth for all subsequent analysis.

Stratum B is the decisive stratum: these are the columns the patch reclassified from mismatch to match. We assess concordance between the consensus human labels and each script using McNemar’s test with Yates’ continuity correction throughout, which evaluates whether disagreements are symmetric or systematically biased in one direction. For Stratum B, the original script disagrees with human judgment on all 20 columns, always in the same direction—labeling columns as mismatches that humans consider equivalent—a bias that is highly statistically significant (McNemar $\chi^2 = 18.05$, $p < 0.0001$). The patched script agrees with human judgment on all 20 Stratum B columns, with no discordant pairs. For Stratum A, both scripts achieve perfect concordance with human labels. For Stratum C, neither script agrees with human consensus on any of the 15 columns. The discordant pairs are nearly balanced ($b = 7$, $c = 8$), yielding McNemar $\chi^2 = 0.00$ ($p = 1.0$) with continuity correction, indicating that neither script exhibits a systematic directional bias

Table 1: Stratified annotation study results (Fleiss’ $\kappa = 0.664$, substantial). Concordance is measured against the majority-vote consensus label via McNemar’s test (1 d.f.). N/A = no discordant pairs; * $p < 0.001$; ** $p < 0.01$. Overall figures are unweighted across strata; per-stratum results are more informative given deliberate oversampling of Stratum B.**

Stratum	Script	Agree	McNemar χ^2
A ($n = 15$)	Original	15/15 (100%)	N/A
	Patched	15/15 (100%)	N/A
B ($n = 20$)	Original	0/20 (0%)	18.05***
	Patched	20/20 (100%)	N/A
C ($n = 15$)	Original	0/15 (0%)	0.00 ($p=1.0$)
	Patched	0/15 (0%)	0.00 ($p=1.0$)
Overall ($n = 50$)	Original	15/50 (30%)	10.31**
	Patched	35/50 (70%)	0.00 ($p=1.0$)

on these hard cases. The 0% agreement on Stratum C reflects the inherent difficulty of these cases: unlike the clear-cut representational equivalences corrected in Stratum B (e.g., boolean normalization, format differences), Stratum C columns involve subtler semantic judgments on which even human annotators do not fully agree. The fact that 7 of these 15 columns are labeled as equivalent by humans but as mismatches by both scripts suggests that our corrections are conservative by design—we address only the false positives for which strong consensus exists, and any residual cases would further strengthen the reported improvement. Over the stratified sample (which oversamples Stratum B to maximize statistical power on the decisive stratum), the patched script agrees with human judgment on 70% of sampled columns versus 30% for the original; note that per-stratum results are more informative than these unweighted aggregates, and its remaining disagreements show no directional bias (McNemar $\chi^2 = 0.00$, $p = 1.0$), in contrast to the original script’s strongly asymmetric error pattern (McNemar $\chi^2 = 10.31$, $p = 0.001$). These results confirm that the patch corrects genuine false positives validated by human judgment, without introducing new systematic errors. Table 1 presents the full per-stratum results.

Discarding Columns with Ground Truth Errors. During the error analysis phase, our auditor flagged 30 columns (spanning 24 tasks) as having incorrect ground truth and proposed revised SQL queries to correct them. Before accepting these revisions, we conduct an independent validation study to assess whether a reliable correction can be established. Three data engineers—with no access to the auditor’s proposed SQL, the original ground truth, or any prior analysis—independently author SQL queries for all 30 columns using only the task specification, source schemas, and source data. Each query is executed and results are compared pairwise among engineers and against the auditor’s proposed values via exact match, after applying the same normalization rules used in ELT-Bench-Verified’s patched evaluation script (boolean normalization, floating-point tolerance, format normalization, NULL equivalence, and order-insensitive comparison). This ensures the

Table 2: Validation study for the 30 columns categorized as having ground truth errors. Three engineers independently produced SQL and results are assessed via direct comparison. Low inter-annotator agreement indicates that the ground truth errors cannot be reliably corrected; low concordance with the auditor’s proposed values confirms no authoritative replacement can be established.

Comparison	Exact-Match (%)
<i>Inter-Annotator Agreement</i>	
Engineer A vs. Engineer B	70.0%
Engineer A vs. Engineer C	56.7%
Engineer B vs. Engineer C	46.7%
Average	57.8%
<i>Auditor Concordance</i>	
Engineer A vs. Auditor	40.0%
Engineer B vs. Auditor	26.7%
Engineer C vs. Auditor	43.3%
Average	36.7%
<i>Majority-Vote Concordance</i>	
Columns with consensus ($\geq 2/3$)	24/30 (80.0%)
Majority vote vs. Auditor	10/24 (41.7%)

comparison is not artificially deflated by the representational equivalences identified earlier in our analysis.

Table 2 presents the results. Pairwise inter-annotator agreement ranges from 46.7% to 70.0%, with an average of 57.8%. This level of disagreement among domain experts with full access to the source data indicates that no unambiguous correction exists for these columns. Concordance with the auditor’s proposed values is lower still, averaging 36.7% across engineers. Majority-vote consensus is available for only 24 of the 30 columns; on the remaining 6 columns, all three engineers produced entirely different results, further underscoring the fundamental ambiguity of these cases. Among the 24 columns with a majority, the consensus agrees with the auditor on only 41.7% of cases. The low inter-annotator agreement is the decisive criterion: it establishes that no single corrected ground truth can be asserted with confidence for these columns, rendering any revision arbitrary. Replacing the original erroneous values with the auditor’s proposals would therefore not constitute a correction but merely a substitution of one unreliable value for another. We consequently discard all 30 columns from the benchmark while retaining all remaining columns belonging to the same data model, amounting to a removal of just 30 out of 2,494 columns (1.2%) with no loss of data model coverage.

Additionally, we conduct an ablation study (Section 5.2.2) to disentangle the contributions of evaluation script refinements and ground truth column removal.

4 AUDIT FINDINGS

Following our structured assessment, we present the quantitative and qualitative findings of our audit. We classify errors along two dimensions: attribution (benchmark vs. agent) and mitigability. We

first present the full error taxonomy, then discuss the quantitative distribution.

4.1 Error Taxonomy

4.1.1 Agent-Attributable Errors. We define agent-attributable errors as discrepancies arising from flawed SQL logic, incorrect data source selection, or semantic misinterpretation. These represent genuine agent errors and should therefore be counted as failures. We categorize such errors into the following classes:

Flawed SQL Logic. Syntactically correct but logically flawed SQL, including: incorrect aggregation methods, erroneous formulas, boolean logic inversions, incorrect window function specifications, and type mismatches.

JOIN Type Errors. Incorrect JOIN type selection causing missing or extraneous rows—typically INNER JOIN excluding valid rows or FULL OUTER JOIN creating spurious combinations.

Wrong Data Source. Queries using inappropriate source tables or columns, including missing required joins or selection of incorrect columns from correct tables.

Domain Knowledge Gaps. Lack of domain-specific understanding required for correct interpretation (e.g., “winnings” interpreted as race victories rather than points earned).

NULL Handling Errors. Incorrect NULL semantics, such as failing to use NULLS FIRST in ordering or misinterpreting implicit NULL meanings.

Key Generation Errors. Primary key generation using different methodology than ground truth, causing row matching failures.

4.1.2 Benchmark-Attributable Errors. We define benchmark-attributable errors as discrepancies arising from deficiencies in benchmark design rather than from agent reasoning. Specifically, these errors stem from ambiguous task specifications, limitations in the evaluation methodology, or incorrect ground truth values, leading to correct agent outputs being incorrectly classified as failures.

Ambiguous Data Model Description. Schema documentation provides unclear, misleading, or insufficient column descriptions that prevent accurate interpretation of intended semantics. For example, a column described as “total amount” might ambiguously refer to pre-tax totals, post-tax totals, or totals including shipping—with ground truth assuming one interpretation without specification.

Evaluation False Positives. Agent outputs and ground truth are semantically equivalent but flagged as mismatched due to evaluation methodology limitations. We identified six subcategories:

- **Actual Match:** Results are functionally identical; discrepancies stem from evaluation script behavior (e.g., floating-point comparison tolerance).
- **Format Mismatch:** Logically correct SQL produces results in alternative valid formats (e.g., 0.15 vs. 15% for percentages).
- **Duplicated Rows in Ground Truth:** Agent SQL is correct; mismatches arise from erroneous duplicate rows in ground truth data.

Table 3: Distribution of error categories across 660 unmatched columns.

Attribution	Error Category	Count	%
Agent	Flawed SQL Logic	221	33.5%
	JOIN Type Errors	157	23.8%
	Wrong Data Source	38	5.8%
	Domain Knowledge Gaps	16	2.4%
	Key Generation Errors	6	0.9%
	NULL Handling Errors	4	0.6%
	<i>Subtotal (Agent)</i>	<i>442</i>	<i>67.0%</i>
Benchmark	Evaluation False Positives	156	23.6%
	Ambiguous Data Model Descr.	32	4.8%
	GT Calculation Errors	30	4.5%
	<i>Subtotal (Benchmark)</i>	<i>218</i>	<i>33.0%</i>
Total		660	100%

- **NULL Representation:** Agent uses different NULL handling (e.g., `COALESCE(column, 0)`) when schema documentation does not specify behavior.
- **Row Ordering:** SQL logic is correct but result ordering differs due to unspecified `ORDER BY` requirements.
- **Other:** Miscellaneous formatting or representation differences.

Ground Truth Calculation Errors. Ground truth contains values that cannot be derived from any reasonable query over the available schema:

- Values do not match any standard interpretation of the task specification.
- Arbitrary or undocumented business rules appear to have been applied.
- Agent results achieve near-perfect matches (e.g., 99.8%) but fail due to anomalous rows with no reasonable explanation.

4.2 Error Distribution

Table 3 presents the distribution of errors across categories for all 660 analyzed columns. Nearly one-third (33.0%) of all column mismatches are benchmark-attributable rather than genuine agent failures.

At the task level, we observe even more striking patterns. Among the 81 tasks that failed transformation evaluation:

- 17.3% (14 tasks) contain **only agent-attributable errors**
- 14.8% (12 tasks) contain **only benchmark-attributable errors**
- 67.9% (55 tasks) contain **both error types**

A total of 67 tasks (82.7% of failed tasks) contain at least one benchmark-attributable error. These flaws misdirect the community by penalizing correct behavior. We highlight two prominent categories:

Evaluation False Positives (23.6%). These occur when an agent’s output is semantically correct but is flagged as a failure due to rigid evaluation scripts. A representative case is the `GNP_GROWTH_RATE`

column in the world database, described simply as “*The GNP growth rate of the country.*” The word “rate” naturally suggests a decimal fraction, and the agent computes exactly that (e.g., 0.0441 for Aruba), yet the ground truth expects the equivalent percentage (4.41). Every one of the 178 non-null values is exactly 100× smaller than the ground truth—the underlying formula is correct, only the output scale differs. This single formatting choice causes a 0% match rate (see Appendix A.2 for details). Such false positives force developers to over-engineer their models to guess arbitrary formatting conventions rather than optimizing for correct logic.

Ground Truth Calculation Errors (4.5%). These are non-mitigable errors where the ground-truth data contains values that cannot be derived from any reasonable query over the available schema. For example, in the `codebase_community` database, a column defined as “set to 1 if the post was posted by the most influential user” has ground truth values that match *only* when “most influential” is operationalized as users with the specific reputation score of 995—ranking 306th out of all users—while users with reputations exceeding 87,000 are labeled as non-influential. This undocumented and counterintuitive rule cannot be deduced from the specification. As detailed in Appendix A.1, we tested five standard operationalizations of “most influential”—highest reputation, most posts, most badges, most profile views, and most upvotes—all of which are far more defensible, yet none reproduce the ground truth. Evaluating agents against such targets fundamentally breaks the utility of the benchmark.

Figure 4 illustrates the overall pipeline outcomes, while Figure 1 shows the error type distribution among failed transformation tasks.

5 ELT-BENCH-VERIFIED

Based on our audit findings, we construct ELT-Bench-Verified—a corrected version of the original benchmark. This section presents the corrections applied by the Corrector pipeline (Section 3.2) and the validation experiments.

5.1 Corrections Applied

Evaluation Script Refinements. We modified the transformation stage evaluation script to address false positive categories identified in our taxonomy:

- Standardized boolean comparisons (“true”/“false” vs. 0/1)
- Implemented appropriate floating-point comparison tolerances
- Added format normalization for percentages and decimals
- Handled NULL representation equivalences
- Made row-order comparisons order-insensitive when no ordering is specified

Ground Truth Column Removal. For the 30 columns (spanning 24 tasks) classified as Ground Truth Calculation Errors, an independent validation study (Section 3.2.2) revealed that human experts themselves disagree on the correct interpretation, with an average pairwise agreement of only 57.8%. Since no reliable correction can be established, we remove these 30 columns from the benchmark entirely—amounting to just 1.2% of all columns (30 out of

Table 4: Agent performance before and after benchmark correction. Both evaluations use SWE-Agent with Claude Sonnet 4.5.

Benchmark	SRDEL	SRDT	Models Passed
ELT-Bench (original)	96%	22.66%	46/203
ELT-Bench-Verified	96%	32.51%	66/203

2,494)—while retaining all remaining columns in each affected data model.

Preservation of Ambiguity. We deliberately did not modify task descriptions for the 32 columns classified as Ambiguous Data Model Description errors. Handling specification ambiguity is a capability that AI agents should possess—potentially through specialized disambiguation agents—and this capability should be evaluated rather than circumvented through benchmark simplification.

5.2 Validation

To validate that our corrections have a measurable effect on evaluation outcomes and are not overfit to a single agent configuration, we conduct three experiments.

5.2.1 End-to-End Re-evaluation. Table 4 presents the results of re-evaluating SWE-Agent with Claude Sonnet 4.5 on ELT-Bench-Verified using the same experimental setup described in Section 2. SRDEL remains at 96%, as the corrections target only the transformation stage. SRDT increases from 22.66% to 32.51%—a 9.85 percentage point gain (43.5% relative improvement), with 20 additional data models now passing evaluation. Importantly, the denominator remains 203 data models: the 30 removed columns are distributed across data models that retain other columns, so no data model is eliminated from the benchmark. The improvement reflects two effects: (i) correcting evaluation false positives that previously penalized correct agent outputs, and (ii) removing unreliable ground truth columns that no agent could match correctly.

5.2.2 Ablation of Correction Strategies. To disentangle the contributions of each correction strategy, we evaluate SWE-Agent with Claude Sonnet 4.5 under three configurations, using the same experimental setup described in Section 2: (i) *evaluation script refinement only*, which applies the patched evaluation logic while retaining the original ground truth and all columns; (ii) *GT column removal only*, which removes the 30 columns with unreliable ground truth while retaining the original evaluation script; and (iii) *ELT-Bench-Verified*, which combines both corrections. Table 5 presents the results.

Evaluation script refinements account for the majority of the improvement (15 additional models), while GT column removal contributes a smaller but complementary gain (4 additional models). The combined correction yields 20 additional models—one more than the sum of individual gains (15+4=19)—indicating that at least one data model contains both evaluation false positives and ground truth errors, requiring both corrections simultaneously to pass.

5.2.3 Multi-Agent Validation. To demonstrate that the corrections are not overfit to a single agent, we evaluate a second agent configuration

Table 5: Ablation of correction strategies. All evaluations use SWE-Agent with Claude Sonnet 4.5. The baseline is the original ELT-Bench evaluation.

Configuration	SRDT	Models Passed
ELT-Bench (original)	22.66%	46/203
+ Evaluation script refinement only	30.05%	61/203
+ GT column removal only	24.63%	50/203
ELT-Bench-Verified (both)	32.51%	66/203

Table 6: Multi-agent validation. Performance of two agent configurations on the original and corrected benchmark.

Agent	Benchmark	SRDEL	SRDT
SWE-Agent w/ Claude Sonnet 4.5	ELT-Bench	96%	22.66%
	ELT-Bench-Verified	96%	32.51%
Baseline (ReAct) w/ Claude Sonnet 4.5	ELT-Bench	98%	20.20%
	ELT-Bench-Verified	98%	32.51%

—a Baseline agent implemented in LangGraph [12] using the ReAct [23] agentic paradigm, paired with Claude Sonnet 4.5—on both the original ELT-Bench and ELT-Bench-Verified. Table 6 presents the results. Both agents show substantial improvement on ELT-Bench-Verified, confirming that the identified errors were genuine benchmark issues rather than artifacts of one agent’s behavior. Notably, both agents converge to the same SRDT (32.51%, 66/203 models) on the corrected benchmark despite differing on the original (22.66%, 46/203 models for SWE-Agent vs. 20.20%, 41/203 models for ReAct). However, the two agents arrive at the same 32.51% SRDT via different paths: ReAct achieves higher SRDEL (98% vs. 96%) and matches more columns overall (1,986 vs. 1,854), yet coincidentally passes the same number of data models (66/203). As shown in Appendix B, the 66 passing models are not identical across the two agents. The fact that both agents improve substantially on ELT-Bench-Verified—despite using different agentic strategies—provides evidence that the corrections address genuine benchmark defects rather than artifacts of a single agent’s behavior.

6 RELATED WORK

ELT Pipelines and Automation. The evolution from ETL to ELT reflects a broader shift toward cloud-native data architectures [19], where elastic compute in platforms such as Snowflake [7] enables transformations to execute directly within the warehouse. This paradigm preserves raw data fidelity, supports schema-on-read flexibility, and reduces the operational overhead of maintaining dedicated transformation infrastructure. However, constructing ELT pipelines remains labor-intensive, requiring expertise in source-specific extraction configuration (e.g., Airbyte [1]), data loading orchestration, and SQL-based transformation logic (e.g., dbt [8]). Recent work has explored AI-driven automation of individual pipeline stages,

but standardized evaluation of these capabilities has lagged behind system development.

Benchmarks for Data Engineering. Existing benchmarks for data-related AI tasks predominantly focus on text-to-SQL evaluation. Widely adopted benchmarks such as BIRD [14] and Spider 2.0 [13] have been central to measuring progress in SQL generation, but they evaluate only the transformation stage in isolation, assuming data is already available in queryable form. The extraction, loading, and tool configuration stages that precede transformation in real ELT workflows fall outside their scope. TPC-DI [15], the industry-standard benchmark for data integration, covers the full ETL workflow but was designed for traditional data integration systems rather than AI agents and follows the ETL (transform-before-load) paradigm rather than modern ELT.

ELT-Bench [10] addresses this gap as the first benchmark specifically designed to evaluate AI agents on end-to-end ELT pipeline construction. By requiring agents to configure extraction from heterogeneous sources, orchestrate loading into Snowflake, and write dbt transformation models, ELT-Bench evaluates capabilities that no prior benchmark captures. Our work builds on ELT-Bench by auditing its data quality and releasing a corrected version.

Annotation Errors and Benchmark Auditing. A growing body of literature has identified systematic quality issues in widely used benchmarks across NLP, software engineering, and data management, including annotation inconsistencies, ambiguous task specifications, and flawed ground truth labels [3, 16]. In the text-to-SQL domain, prior analyses have reported annotation errors in BIRD [21], and a recent comprehensive audit demonstrated that error rates in BIRD Mini-Dev and Spider 2.0-Snow can exceed 50%, substantially distorting leaderboard rankings [9].

To address similar challenges, prior work has combined automated validation techniques with expert review, using generated test cases, oracle construction, and execution-based validation to strengthen evaluation rigor. Our methodology follows this paradigm: we employ automated LLM-based root cause analysis at scale (Phase 2) but require comprehensive human validation of all findings (Phase 3). Our results extend the observation of pervasive benchmark-attributable errors from text-to-SQL settings to the more complex domain of end-to-end ELT pipeline evaluation.

7 DISCUSSION

7.1 The Evolving Landscape of ELT Automation

The progression from 1% to 22.66% to 32.51% SRDT reflects two compounding factors that obscured agent capabilities: the use of an older model in the original evaluation, and benchmark quality issues. The first gap is attributable to model improvement (with the agent framework held constant), while the second is attributable entirely to benchmark correction. At 96% SRDEL, the extraction and loading stage is largely solved, suggesting that future benchmark development should focus on more challenging pipeline stages.

7.2 Implications for Benchmark Design

Our findings extend recent observations of pervasive annotation errors in text-to-SQL benchmarks [9] to end-to-end ELT pipeline evaluation. These results reinforce the need for systematic quality

auditing as a standard practice in benchmark development, particularly for agentic tasks where evaluation involves multi-step pipelines and execution-based correctness checking.

7.3 Implications for Agentic ELT Systems

Our error taxonomy provides a detailed map of where current AI agents succeed and fail in ELT pipeline construction, offering actionable guidance for practitioners building agentic data engineering systems.

What agents can already do. The extraction and loading stage is effectively solved: at 96% SRDEL, current models can reliably interpret connection configurations, generate Terraform definitions for Airbyte connectors, and orchestrate data loading into Snowflake with minimal failure. For straightforward transformation tasks—those involving direct column mappings, simple aggregations, and well-specified schemas—agents also perform reliably, as evidenced by the 1,854 (SWE-Agent) and 1,986 (ReAct) columns matched correctly out of 2,202 eligible. Practitioners can confidently deploy agents for the EL stage and for transformation tasks with clear, unambiguous specifications.

Where agents struggle: recurring error patterns. Our taxonomy reveals that agent failures are heavily concentrated in two categories: *Flawed SQL Logic* (33.5% of all errors) and *JOIN Type Errors* (23.8%), which together account for 57.3% of all agent-attributable failures. Importantly, these are not failures of task comprehension—agents generally understand *what* to compute—but rather failures of SQL construction. By contrast, domain knowledge gaps account for only 2.4% of errors, indicating that semantic understanding is rarely the bottleneck. A detailed examination of the 660 column-level errors reveals several recurring patterns:

- **Aggregation with ties.** A frequent source of errors is the use of `LIMIT 1` to retrieve a superlative (e.g., “the film with the highest replacement cost”). When multiple entities share the same maximum value, `LIMIT 1` returns an arbitrary result. This pattern recurs across databases and accounts for a notable share of Flawed SQL Logic errors.
- **NULL handling assumptions.** Agents often make incorrect assumptions about NULL semantics: replacing NULLs with zeros when the ground truth preserves them, excluding NULLs from ratio computations when they should be included, or using `COALESCE(..., 'Null')` (a string literal) instead of preserving actual NULL values. In some cases, the same database uses different NULL conventions across columns, making consistent handling particularly challenging.
- **String fidelity and data-value assumptions.** Agents frequently apply implicit normalization—trimming whitespace, altering capitalization, or reformatting values—when the ground truth expects source data to be preserved exactly. We observe whitespace-related errors across 12 columns spanning 8 databases. Similarly, agents assume standard representations for real-world entities that differ from the actual data (e.g., filtering for ‘USA’ when the data uses ‘United States’, or missing a brand name due to punctuation differences).

Strategies for improving agentic ELT systems. The concentration of errors in SQL mechanics rather than task comprehension suggests several high-leverage strategies:

- **Execution-based self-verification.** Agents should execute their generated queries and verify that output schema, row counts, and value distributions match expectations. Our Auditor pipeline demonstrates feasibility: the LLM agent self-validated corrected SQL for 630 of 660 columns. Agents should specifically check for unexpected row count changes after JOINS (a signal for incorrect join types, which alone account for 23.8% of failures) and verify that superlative queries handle ties correctly.
- **Data profiling before query generation.** Many errors stem from agents assuming data formats without inspecting actual values. A preliminary profiling step—sampling column values to detect whitespace patterns, NULL rates, string formats, and numeric representations—could prevent a substantial class of errors (e.g., revealing that a boolean column uses 0/1 rather than True/False, or that country names use full forms rather than abbreviations).
- **Multi-agent portfolios.** SWE-Agent and ReAct share only 14 of their fully correct databases, with each exclusively solving additional tasks the other cannot (Section 5.2.3). This complementarity suggests that ensemble approaches—running multiple agents and selecting the best output per task—could substantially improve coverage.
- **Human-in-the-loop for ambiguity resolution.** Ambiguous specifications account for 4.8% of column errors, where agents select a defensible but incorrect interpretation (e.g., “the most common film genre” without tie-breaking behavior). Multi-agent debate architectures, where several agents independently propose interpretations and a human resolves disagreements, are a promising direction for production ELT systems.

What remains hard. Even on ELT-Bench-Verified, 67.5% of data models still fail. The hardest cases involve the interplay of multiple error types within a single task—a column requiring both a correct multi-table join *and* proper NULL handling *and* data-aware string matching. These compounding difficulties, combined with domain-specific business logic and long chains of dependent SQL operations, suggest that future progress will require advances in structured reasoning over relational schemas and data-aware query generation.

7.4 Limitations

Our analysis has several limitations. First, our Auditor-Corrector pipeline was applied to the failed tasks of a single agent configuration (SWE-Agent with Claude Sonnet 4.5). Different agents may fail on different tasks, potentially revealing additional benchmark-attributable errors not captured by our audit. While we validate the corrected benchmark with a second agent (Section 5.2.3), applying the full Auditor pipeline to the failures of multiple agents would provide broader coverage; however, as discussed in Section 2, each full evaluation run requires multiple days and hundreds of dollars in API costs, making exhaustive multi-agent auditing prohibitively expensive. Second, the primary categorization of all 660 columns

Table 7: Alternative interpretations of “most influential user” tested against the ground truth (91,966 posts total). No standard interpretation reproduces the GT; only an undocumented filter (Reputation = 995) achieves 100%.

Interpretation	Selected User(s)	Match
Most posts	User 805 (1,720 posts)	98.07%
Highest reputation	User 919 (rep. 87,393)	98.63%
Most badges	User 919	98.63%
Most profile views	User 919 (20,932 views)	98.63%
Most upvotes	User 1036	99.74%
Reputation = 995	Users 3432 & 8165	100%

into our error taxonomy was performed by a single data engineer. An inter-annotator agreement study on a random sample of 50 columns yields Fleiss’ $\kappa = 0.851$ (almost perfect) for the high-level agent-vs.-benchmark attribution and $\kappa = 0.755$ (substantial) for the exact 14-category classification, indicating high reproducibility. However, the study covers a sample rather than the full set of 660 columns, and edge cases in the complete dataset may exhibit lower consistency. Third, our error attribution required judgment calls in ambiguous cases. In particular, the boundary between “Ambiguous Data Model Description” (preserved in the benchmark) and “Evaluation False Positive” (corrected) involves a judgment about whether the specification is genuinely unclear or merely admits representational variation—reasonable annotators might draw this line differently.

8 CONCLUSION

We presented a comprehensive reassessment of ELT-Bench revealing that model improvement (SRDEL: 37%→96%, SRDT: 1%→22.66%) and benchmark quality issues (SRDT: 22.66%→32.51% after correction) jointly led the original evaluation to substantially underestimate agent capabilities. Our systematic audit found that 82.7% of failed tasks contain benchmark-attributable errors, and we release ELT-Bench-Verified as a corrected benchmark. While transformation remains challenging—67.5% of data models still fail—our error taxonomy and analysis of recurring failure patterns provide actionable guidance for building more effective agentic ELT systems.

A ILLUSTRATIVE EXAMPLES

A.1 Ground Truth Error

We illustrate the nature of ground truth calculation errors with a concrete example from the `codebase_community` database. The column `POSTED_BY_MOST_INFLUENTIAL_USER` is described as: “Set to 1 if the post was posted by the most influential user, otherwise, set to 0.”

The term “most influential user” is inherently ambiguous—it could reasonably refer to the user with the highest reputation, the most posts, the most upvotes, or other engagement metrics. Table 7 lists five standard interpretations, each producing a different user and a different match rate against the ground truth.

The ground truth marks exactly 56 posts as being from the “most influential” user. These 56 posts belong to two users (3432 and

Table 8: User comparison for the POSTED_BY_MOST_INFLUENTIAL_USER column. The GT flags only users with Reputation=995 as “most influential,” ignoring users with far higher reputation, post counts, and engagement metrics.

User	Rep.	Posts	UpVotes	Views	GT=1
805	65,272	1,720	7,035	5,680	0
919	87,393	1,204	11,273	20,932	0
3432	995	22	79	82	22
8165	995	34	66	81	34

Table 9: Format mismatch for GNP_GROWTH_RATE. The agent’s formula is correct; only the output scale differs. All 178 non-null values exhibit an exact 100× ratio.

Country	GT (%)	Pred (decimal)	Ratio
Aruba	4.41	0.0441	100.0
Angola	−16.73	−0.1673	100.0
Albania	28.20	0.2820	100.0
UAE	3.04	0.0304	100.0
Argentina	5.24	0.0524	100.0

8165), both with a reputation score of exactly 995—placing them at rank 306–307 out of all users (99.24th percentile). Meanwhile, users with reputations of 65,272 and 87,393 are labeled as non-influential. Table 8 highlights this contrast.

While the column description is genuinely ambiguous—a reasonable point of discussion—the ground truth error is unambiguous: no standard definition of “influence” would select users ranked 306th by reputation over users ranked 1st or 2nd. The GT was evidently generated using an undocumented filter (`WHERE Reputation = 995`) that bears no relationship to the stated specification. This column is one of the 30 removed from ELT-Bench-Verified.

A.2 Evaluation False Positive

We illustrate evaluation false positives with the `GNP_GROWTH_RATE` column in the world database, described as: “*The GNP growth rate of the country.*”

The agent applies the standard growth-rate formula ($GNP - GNP_{old} / GNP_{old}$), returning a decimal fraction (e.g., 0.0441 for a 4.41% growth). The ground truth stores the same quantity as a percentage (4.41). The two representations are mathematically equivalent—every value differs by exactly a factor of 100—yet the evaluation script performs an exact string comparison, yielding a 0% match rate across all 178 non-null countries.

Table 9 shows representative rows. The ratio between ground truth and predicted values is exactly 100.0 for every row (standard deviation ≈ 0). The column description uses the word “rate,” which in standard mathematical usage denotes a ratio (0–1), not a percentage (0–100). The agent’s interpretation is arguably more faithful to the specification than the ground truth, yet it receives a 0% score. Multiplying by 100 yields a perfect 178/178 match, confirming that the mismatch is purely representational.

Table 10: Detailed comparison of agent performance on ELT-Bench-Verified. Both agents pass 66/203 data models but differ in column-level behavior and fully correct databases.

Metric	SWE-Agent	ReAct
SRDEL	96%	98%
SRDT	32.51%	32.51%
Eligible data models	194	199
Correct data models	66	66
Matched columns	1,854	1,986
Unmatched columns	348	411
Fully correct databases	18	15
Shared fully correct DBs	14	
SWE-Agent only	4	
ReAct only	1	

B MULTI-AGENT VALIDATION DETAIL

Table 10 provides a detailed comparison of SWE-Agent and Baseline (ReAct) on ELT-Bench-Verified. Although both agents pass 66 data models (32.51% SRDT), the sets of fully correct databases differ. SWE-Agent achieves perfect scores on 18 databases, while ReAct achieves perfect scores on 15. The two agents share 14 fully correct databases; SWE-Agent exclusively solves 4 additional databases (`instagram_business`, `law_episode`, `linkedin`, `menu`), while ReAct exclusively solves 1 (`bike_share_1`). This confirms that the identical SRDT is coincidental: SWE-Agent’s correct models are more concentrated within fewer databases, while ReAct’s are more evenly distributed across a broader set of tasks.

USE OF AI

The authors used AI-assisted writing tools (e.g., large language models) for grammar checking and improving the clarity of the manuscript. All content was written and reviewed by the authors, who take full responsibility for the work.

REFERENCES

- [1] Airbyte. 2025. Airbyte. <https://airbyte.com/>
- [2] Ayorinde Olayiwola Akindemowo, Eseoghene Daniel Erigha, Ehimah Obuse, Joshua Oluwagbenga Ajayi, Ayobami Adebayo, Afeez A Afuwape, and Anthonette Adanyin. 2021. A Conceptual Framework for Automating Data Pipelines Using ELT Tools in Cloud-Native Environments. *Journal of Frontiers in Multidisciplinary Research* 2, 1 (2021), 440–452.
- [3] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. 2024. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992 [cs.SE] <https://arxiv.org/abs/2410.06992>
- [4] Anthropic. 2024. The Claude Model Family: Claude 3.5 Sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>
- [5] Anthropic. 2025. Introducing Claude Opus 4.5. <https://www.anthropic.com/news/claude-opus-4-5>
- [6] Anthropic. 2025. Introducing Claude Sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>
- [7] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. (2016), 215–226. <https://doi.org/10.1145/2882903.2903741>
- [8] dbt Labs. 2025. dbt. <https://www.getdbt.com/>
- [9] Tengjun Jin, Yoojin Choi, Yuxuan Zhu, and Daniel Kang. 2026. Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards. arXiv:2601.08778 [cs.AI] <https://arxiv.org/abs/2601.08778>

- [10] Tengjun Jin, Yuxuan Zhu, and Daniel Kang. 2025. ELT-Bench: An End-to-End Benchmark for Evaluating AI Agents on ELT Pipelines. *Proc. VLDB Endow.* 19, 2 (2025), 84–98. <https://www.vldb.org/pvldb/vol19/p84-jin.pdf>
- [11] Soma Sundar Reddy Kancharla. 2025. ETL vs ELT: Evolving Approaches to Data Integration. *Journal Of Engineering And Computer Sciences* 4, 7 (2025), 1065–1074.
- [12] LangChain, Inc. [n.d.]. LangGraph: Low-level orchestration framework for building stateful agents. <https://github.com/langchain-ai/langgraph> Accessed: 2026-03-20.
- [13] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv:2411.07763 [cs.CL] <https://arxiv.org/abs/2411.07763>
- [14] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2024. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc.
- [15] Meikel Poess, Tilmann Rabl, Hans-Arno Jacobsen, and Brian Caulfield. 2014. TPC-DI: the first industry benchmark for data integration. *Proc. VLDB Endow.* 7, 13, 1367–1378. <https://doi.org/10.14778/2733004.2733009>
- [16] Mohammadreza Pourreza and Davood Rafiei. 2023. Evaluating Cross-Domain Text-to-SQL Models and Benchmarks. arXiv:2310.18538 [cs.CL] <https://arxiv.org/abs/2310.18538>
- [17] Aiswarya Raj, Jan Bosch, Helena Holmström Olsson, and Tian J. Wang. 2020. Modelling Data Pipelines. In *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 13–20. <https://doi.org/10.1109/SEAA51224.2020.00014>
- [18] Vladislav Shkapenyuk, Divesh Srivastava, Theodore Johnson, and Parisa Ghane. 2025. Automatic Metadata Extraction for Text-to-SQL. arXiv:2505.19988 [cs.DB] <https://arxiv.org/abs/2505.19988>
- [19] Alkis Simitsis, Spiros Skiadopoulos, and Panos Vassiliadis. 2023. The History, Present, and Future of ETL Technology. (2023), 3–12. <https://ceur-ws.org/Vol-3369/invited1.pdf>
- [20] Pengfei Wang, Baolin Sun, Xuemei Dong, Yaxun Dai, Hongwei Yuan, Mengdie Chu, Yingqi Gao, Xiang Qi, Peng Zhang, and Ying Yan. 2025. Agentar-Scale-SQL: Advancing Text-to-SQL through Orchestrated Test-Time Scaling. arXiv:2509.24403 [cs.CL] <https://arxiv.org/abs/2509.24403>
- [21] Niklas Wretblad, Fredrik Gordh Riseby, Rahul Biswas, Amin Ahmadi, and Oskar Holmström. 2024. Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark. arXiv:2402.12243 [cs.CL] <https://arxiv.org/abs/2402.12243>
- [22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793 [cs.SE] <https://arxiv.org/abs/2405.15793>
- [23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>